Mock Test — Solutions

C Programming Fundamentals: 25 questions, worked step by step

CPE101 Teaching Assistant

CPE101 Computer Programming
King Mongkut's University of Technology Thonburi (KMUTT)

Tutorial session

How to use these slides

- 25 questions $\times$ 4 marks, grouped by topic (not in exam order).
- Each question appears first. **Try it yourself**, then click to reveal the answer.
- Every program here was compiled with `gcc -Wall -Wextra -std=c11 … -lm` and every output shown is from a **real run**.
- Source code: `03-mock-solutions/examples/qNN_*.c`

TIP Try it yourself before revealing the answer, then compare with the reasoning on the slide.

Topic	Questions
Program structure	1, 2
Output and input	3, 5, 8, 9, 10
Types and constants	4, 6, 7
Formulas	11, 12
Decisions	13, 14, 15, 16
Loops	17 – 22
Operators, arrays	23, 24
Series	25

Q1 · The basic structure of a C program

Question. Describe the basic structure of a C program. Explain the role of each section: documentation, link, definition, global declaration, `main( )`, and sub-program (user-defined function).

Section	Contains	Role

Q1 · The basic structure of a C program

Question. Describe the basic structure of a C program. Explain the role of each section: documentation, link, definition, global declaration, `main( )`, and sub-program (user-defined function).

Section	Contains	Role
Documentation	comments `/* ... */`	program name, author, purpose. Ignored by the compiler

Q1 · The basic structure of a C program

Question. Describe the basic structure of a C program. Explain the role of each section: documentation, link, definition, global declaration, `main( )`, and sub-program (user-defined function).

Section	Contains	Role
Documentation	comments `/* ... */`	program name, author, purpose. Ignored by the compiler
Link	`#include <stdio.h>`	brings in library declarations (`printf`, `sqrt`, ...)

Q1 · The basic structure of a C program

Question. Describe the basic structure of a C program. Explain the role of each section: documentation, link, definition, global declaration, `main( )`, and sub-program (user-defined function).

Section	Contains	Role
Documentation	comments `/* ... */`	program name, author, purpose. Ignored by the compiler
Link	`#include <stdio.h>`	brings in library declarations (`printf`, `sqrt`, ...)
Definition	`#define PI 3.14159`	symbolic constants, replaced by the preprocessor

Q1 · The basic structure of a C program

Question. Describe the basic structure of a C program. Explain the role of each section: documentation, link, definition, global declaration, `main( )`, and sub-program (user-defined function).

Section	Contains	Role
Documentation	comments `/* ... */`	program name, author, purpose. Ignored by the compiler
Link	`#include <stdio.h>`	brings in library declarations (`printf`, `sqrt`, ...)
Definition	`#define PI 3.14159`	symbolic constants, replaced by the preprocessor
Global declaration	global variables, function prototypes	names usable by **every** function in the file

Q1 · The basic structure of a C program

Question. Describe the basic structure of a C program. Explain the role of each section: documentation, link, definition, global declaration, `main( )`, and sub-program (user-defined function).

Section	Contains	Role
Documentation	comments `/* ... */`	program name, author, purpose. Ignored by the compiler
Link	`#include <stdio.h>`	brings in library declarations (`printf`, `sqrt`, ...)
Definition	`#define PI 3.14159`	symbolic constants, replaced by the preprocessor
Global declaration	global variables, function prototypes	names usable by **every** function in the file
`main()`	`int main(void) { ... }`	**required**; execution starts and ends here

Q1 · The basic structure of a C program

Question. Describe the basic structure of a C program. Explain the role of each section: documentation, link, definition, global declaration, `main()`, and sub-program (user-defined function).

Section	Contains	Role
Documentation	comments `/* ... */`	program name, author, purpose. Ignored by the compiler
Link	`#include <stdio.h>`	brings in library declarations (`printf`, `sqrt`, ...)
Definition	`#define PI 3.14159`	symbolic constants, replaced by the preprocessor
Global declaration	global variables, function prototypes	names usable by **every** function in the file
`main()`	`int main(void) { ... }`	**required**; execution starts and ends here
Sub-program	user-defined functions	smaller tasks called from `main()`

Q1 · One program with all six sections

```c
/*  Documentation section
 *  Program : Area of a circle
 *  Author  : CPE101 TA
 */
#include <stdio.h>          /* Link section       */

#define PI 3.14159          /* Definition section */

double area(double r);      /* Global declaration */
int call_count = 0;

int main(void)              /* main() section     */
{
    double r = 2.0;
    printf("Area = %.2f\n", area(r));
    printf("area() was called %d time(s)\n", call_count);
    return 0;
}

double area(double r)       /* Sub-program section */
{
    call_count++;
    return PI * r * r;
}
```

```
Area = 12.57
area() was called 1 time(s)
```

- Only `main()` is compulsory. The others are used when needed.
- The prototype in the global section lets `main()` call `area()` **before** its body appears.
- `call_count` is global, so both functions can use it.

TIP Every program must have `main()`. Include the other parts only as needed.

Q2 · Multi-file programs, `extern` and `static`

Question. What is a multi-file program? Describe the role of header files and the `extern` keyword when a function or global variable defined in one `.c` file is used in another. How does `static` limit this access?

Q2 · Multi-file programs, `extern` and `static`

Question. What is a multi-file program? Describe the role of header files and the `extern` keyword when a function or global variable defined in one `.c` file is used in another. How does `static` limit this access?

- **Multi-file program**: the source is split over several `.c` files, each compiled separately (`gcc -c`) and then **linked** into one executable.

Q2 · Multi-file programs, `extern` and `static`

Question. What is a multi-file program? Describe the role of header files and the `extern` keyword when a function or global variable defined in one `.c` file is used in another. How does `static` limit this access?

- **Multi-file program**: the source is split over several `.c` files, each compiled separately (`gcc -c`) and then **linked** into one executable.
- **Header file** (`.h`): holds *declarations*, i.e. prototypes and `extern` variables. Every `.c` file that `#includes` it agrees on the same names and types. Protect it with an include guard.

Q2 · Multi-file programs, `extern` and `static`

Question. What is a multi-file program? Describe the role of header files and the `extern` keyword when a function or global variable defined in one `.c` file is used in another. How does `static` limit this access?

- **Multi-file program**: the source is split over several `.c` files, each compiled separately (`gcc -c`) and then **linked** into one executable.
- **Header file** (`.h`): holds *declarations*, i.e. prototypes and `extern` variables. Every `.c` file that `#includes` it agrees on the same names and types. Protect it with an include guard.
- **`extern int total;`**: "`total` is *defined in another file*". No memory is created. Exactly **one** file has the real definition `int total = 0;`. Functions are `extern` by default, so a prototype is enough.

Q2 · Multi-file programs, `extern` and `static`

Question. What is a multi-file program? Describe the role of header files and the `extern` keyword when a function or global variable defined in one `.c` file is used in another. How does `static` limit this access?

- **Multi-file program**: the source is split over several `.c` files, each compiled separately (`gcc -c`) and then **linked** into one executable.
- **Header file** (`.h`): holds *declarations*, i.e. prototypes and `extern` variables. Every `.c` file that `#includes` it agrees on the same names and types. Protect it with an include guard.
- **`extern int total;`**: "`total` is *defined in another file*". No memory is created. Exactly **one** file has the real definition `int total = 0;`. Functions are `extern` by default, so a prototype is enough.
- **`static`** on a global variable or function: **internal linkage**. The name is visible *only inside its own file*. Another file that tries to use it gets a link error (`undefined reference`).

Q2 · Example

counter.h

```h
#ifndef COUNTER_H
#define COUNTER_H

extern int total;
void add(int x);

#endif
```

counter.c

```c
#include "counter.h"

int total = 0;         /* definition */
static int calls = 0; /* private    */

void add(int x)
{
    calls++;
    total += x;
}
```

main.c

```c
#include <stdio.h>
#include "counter.h"

int main(void)
{
    add(5);
    add(7);
    printf("total = %d\n", total);
    /* calls is NOT usable here */
    return 0;
}
```

```
$ gcc -Wall -Wextra -std=c11 main.c counter.c -o q02
$ ./q02
total = 12
```

TIP `extern` = “exists in another file”; `static` = “usable only in this file” (see the Compiler slides for details).

Q3 · A pattern with `printf()` only

Question. Write a C program that uses only `printf()` statements to display the pattern (two centred lines): `WELCOME  TO / C  LAB`

Q3 · A pattern with `printf()` only

Question. Write a C program that uses only `printf()` statements to display the pattern (two centred lines): `WELCOME  TO` / `C  LAB`

```c
#include <stdio.h>

int main(void)
{
    printf("WELCOME TO\n");
    printf("  C LAB\n");
    return 0;
}
```

```
WELCOME  TO
   C  LAB
```

- Each line ends with `\n`.
- `WELCOME  TO` is 10 characters and `C  LAB` is 5, so we need $(10-5)/2 \approx 2$ leading spaces to centre it.
- Keep in mind that in this case, it is not perfectly centered.

Q3 · A pattern with `printf()` only

Question. Write a C program that uses only `printf()` statements to display the pattern (two centred lines): `WELCOME  TO / C  LAB`

```c
#include <stdio.h>

int main(void)
{
    printf("WELCOME TO\n");
    printf("  C LAB\n");
    return 0;
}
```

```
WELCOME  TO
   C  LAB
```

- Each line ends with \n.
- `WELCOME  TO` is 10 characters and `C  LAB` is 5, so we need $(10-5)/2 \approx 2$ leading spaces to centre it.
- Keep in mind that in this case, it is not perfectly centered.

Tip: can't see how many spaces you printed? Temporarily print * instead: "***C**LAB".

Q5 · Escape sequences: what and why

Question. What is an escape sequence? Why does C need escape sequence characters? Explain any five escape sequences (e.g. `\n`, `\t`, `\b`, `\"`, `\a`) with one `printf()` example each.

TIP Escape sequences stand in for characters you cannot type directly in a string, such as a newline or the " character.

Q5 · Escape sequences: what and why

Question. What is an escape sequence? Why does C need escape sequence characters? Explain any five escape sequences (e.g. `\n`, `\t`, `\b`, `\"`, `\a`) with one `printf()` example each.

- An **escape sequence** is a backslash `\` followed by a character. Together they stand for **one** special character.

TIP Escape sequences stand in for characters you cannot type directly in a string, such as a newline or the `"` character.

Q5 · Escape sequences: what and why

Question. What is an escape sequence? Why does C need escape sequence characters? Explain any five escape sequences (e.g. `\n`, `\t`, `\b`, `\"`, `\a`) with one `printf()` example each.

- An **escape sequence** is a backslash \ followed by a character. Together they stand for **one** special character.
- **Why needed?** Some characters can't be typed directly inside "...":
  - **invisible / control** characters: new line, tab, backspace, beep
  - characters that have a **special meaning** in a string: " ends the string, \ starts an escape

TIP Escape sequences stand in for characters you cannot type directly in a string, such as a newline or the " character.

Q5 · Escape sequences: what and why

Question. What is an escape sequence? Why does C need escape sequence characters? Explain any five escape sequences (e.g. `\n`, `\t`, `\b`, `\"`, `\a`) with one `printf()` example each.

- An **escape sequence** is a backslash `\` followed by a character. Together they stand for **one** special character.
- **Why needed?** Some characters can't be typed directly inside `"..."`:
  - **invisible / control** characters: new line, tab, backspace, beep
  - characters that have a **special meaning** in a string: `"` ends the string, `\` starts an escape
- The compiler turns `\n` into the single new-line character (ASCII 10). `"A\n"` is 2 characters, not 3.

TIP Escape sequences stand in for characters you cannot type directly in a string, such as a newline or the `"` character.

Q5 · Five (+1) escape sequences in action

```c
printf("Line 1\nLine 2\n");     /* \n  new line       */
printf("Name\tScore\n");        /* \t  horizontal tab */
printf("ABC\bD\n");             /* \b  backspace      */
printf("He said \"Hi\"\n");     /* \"  double quote   */
printf("Done!\a\n");            /* \a  alert (beep)   */
printf("C:\\cpe101\\lab1\n");   /* \\  backslash      */
```

\n	move to the start of the next line
\t	jump to the next tab stop
\b	move the cursor back one place
\”	a literal ” inside a string
\a	ring the terminal bell (nothing visible)
\\	a literal backslash

On screen:

```
Line 1
Line 2
Name    Score
ABD
He said "Hi"
Done!
C:\cpe101\lab1
```

Q5 · Five (+1) escape sequences in action

```c
printf("Line 1\nLine 2\n");     /* \n  new line       */
printf("Name\tScore\n");        /* \t  horizontal tab */
printf("ABC\bD\n");             /* \b  backspace      */
printf("He said \"Hi\"\n");     /* \"  double quote   */
printf("Done!\a\n");            /* \a  alert (beep)   */
printf("C:\\cpe101\\lab1\n");   /* \\  backslash      */
```

\n	move to the start of the next line
\t	jump to the next tab stop
\b	move the cursor back one place
\"	a literal " inside a string
\a	ring the terminal bell (nothing visible)
\\	a literal backslash

On screen:

```
Line 1
Line 2
Name    Score
ABD
He said "Hi"
Done!
C:\cpe101\lab1
```

ABC\bD: the cursor steps back over C, then D is written on top of it ⇒ ABD.

\a makes a beep (if sound is on), but prints nothing.

Q8 · scanf with field widths

Question. `scanf("%2d%*1d%3d%d", &a, &b, &c);` The user enters `9876543210`. Find a, b, c. Explain the `*` in `%*1d`. What if the input were `98 7 654 3`?

9	8	7	6	5	4	3	2	1	0

Q8 · scanf with field widths

Question. `scanf("%2d%*1d%3d%d", &a, &b, &c);` The user enters `9876543210`. Find a, b, c. Explain the `*` in `%*1d`. What if the input were `98 7 654 3`?



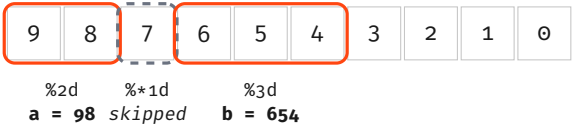
Q8 · scanf with field widths

Question. `scanf("%2d%*1d%3d%d", &a, &b, &c);` The user enters `9876543210`. Find a, b, c. Explain the `*` in `%*1d`. What if the input were `98 7 654 3`?



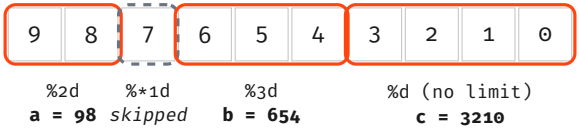
Q8 · scanf with field widths

Question. `scanf("%2d%*1d%3d%d", &a, &b, &c);` The user enters `9876543210`. Find a, b, c. Explain the `*` in `%*1d`. What if the input were `98 7 654 3`?



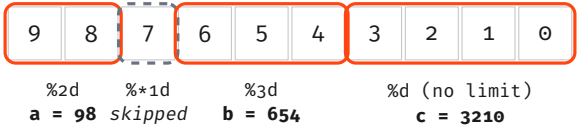
Q8 · scanf with field widths

Question. `scanf("%2d%*1d%3d%d", &a, &b, &c);` The user enters `9876543210`. Find a, b, c. Explain the `*` in `%*1d`. What if the input were `98 7 654 3`?



Q8 · scanf with field widths

Question. `scanf("%2d%*1d%3d%d", &a, &b, &c);` The user enters `9876543210`. Find a, b, c. Explain the `*` in `%*1d`. What if the input were `98 7 654 3`?



- A number in `%2d` is the **maximum width**: read at most 2 characters.
- `*` = **assignment suppression**: read the field, then throw it away. It needs **no** variable in the list.

```
a = 98, b = 654, c = 3210
```

Q8 · And with spaces: 98 7 654 3

%d skips any **whitespace** (spaces, tabs, newlines) *before* it starts reading. The width then counts only digits.

Spec	Reads	Result
`%2d`	`98`	`a = 98`

Q8 · And with spaces: 98 7 654 3

%d skips any **whitespace** (spaces, tabs, newlines) *before* it starts reading. The width then counts only digits.

Spec	Reads	Result
`%2d`	`98`	`a = 98`
`%*1d`	skips space, reads `7`	discarded

Q8 · And with spaces: 98 7 654 3

%d skips any **whitespace** (spaces, tabs, newlines) *before* it starts reading. The width then counts only digits.

Spec	Reads	Result
`%2d`	`98`	`a = 98`
`%*1d`	skips space, reads `7`	discarded
`%3d`	skips space, reads `654`	`b = 654`

Q8 · And with spaces: 98 7 654 3

%d skips any **whitespace** (spaces, tabs, newlines) *before* it starts reading. The width then counts only digits.

Spec	Reads	Result
%2d	98	a = 98
%*1d	skips space, reads 7	discarded
%3d	skips space, reads 654	b = 654
%d	skips space, reads 3	c = 3

```
$ echo "98 7 654 3" | ./q08_scanf
a = 98, b = 654, c = 3
```

Same a and b. Only c changes: 3210 → 3, because %d stops at the end of the input.

Here the spaces happen to match the field widths. A width never makes scanf read *past* a space inside a number.

TIP %2d = read at most 2 digits; %*1d = read 1 digit and discard it; %d skips leading whitespace automatically.

Q9 · `getchar()` / `putchar()`: vowels and spaces

Question. Use `getchar()` to read characters until a full stop (`.`). Count (a) vowels and (b) spaces. Echo each character in uppercase with `putchar()`.

Plan

Q9 · `getchar()` / `putchar()`: vowels and spaces

Question. Use `getchar()` to read characters until a full stop (`.`). Count (a) vowels and (b) spaces. Echo each character in uppercase with `putchar()`.

Plan

1. Loop: read one character with `ch = getchar()`; stop when `ch == '.'` (also stop at EOF, end of input).

Q9 · `getchar()` / `putchar()`: vowels and spaces

Question. Use `getchar()` to read characters until a full stop (`.`). Count (a) vowels and (b) spaces. Echo each character in uppercase with `putchar()`.

Plan

1. Loop: read one character with `ch = getchar()`; stop when `ch == '.'` (also stop at EOF, end of input).
2. If `ch` is a vowel (upper or lower case) $\rightarrow$ `vowels++`. If `ch == ' '` $\rightarrow$ `spaces++`.

Q9 · `getchar()` / `putchar()`: vowels and spaces

Question. Use `getchar()` to read characters until a full stop (`.`). Count (a) vowels and (b) spaces. Echo each character in uppercase with `putchar()`.

Plan

1. Loop: read one character with `ch = getchar()`; stop when `ch == '.'` (also stop at EOF, end of input).
2. If `ch` is a vowel (upper or lower case) $\rightarrow$ `vowels++`. If `ch == ' '` $\rightarrow$ `spaces++`.
3. Echo it: `putchar(toupper(ch))`. `toupper` is in `<ctype.h>` and leaves non-letters unchanged.

Q9 · `getchar()` / `putchar()`: vowels and spaces

Question. Use `getchar()` to read characters until a full stop (`.`). Count (a) vowels and (b) spaces. Echo each character in uppercase with `putchar()`.

Plan

1. Loop: read one character with `ch = getchar()`; stop when `ch == '.'` (also stop at EOF, end of input).
2. If `ch` is a vowel (upper or lower case) → `vowels++`. If `ch == ' '` → `spaces++`.
3. Echo it: `putchar(toupper(ch))`. `toupper` is in `<ctype.h>` and leaves non-letters unchanged.
4. After the loop, print both counts.

TIP `getchar()` reads one character at a time (spaces included), unlike `scanf("%s")`, which stops at whitespace.

Q9 · Solution

```c
#include <stdio.h>
#include <ctype.h>      /* tolower, toupper */
int main(void)
{
    int ch, vowels = 0, spaces = 0;
    printf("Type a sentence ending with '.':\n");
    while ((ch = getchar()) != '.' && ch != EOF)
    {
        switch (tolower(ch))
        {
            case 'a': case 'e': case 'i':
            case 'o': case 'u':
                vowels++;
                break;
            case ' ':
                spaces++;
                break;
        }
        putchar(toupper(ch));
    }
    printf("\nVowels: %d\nSpaces: %d\n", vowels, spaces);
    return 0;
}
```

```
Type a sentence ending with '.':
Hello World, I love C. extra
HELLO WORLD, I LOVE C
Vowels: 6
Spaces: 4
```

- `ch` is an `int`, because `getchar()` returns `int` (so it can also return `EOF`).
- `tolower` lets one `case` list catch `'A'` and `'a'`.
- Text after the `.` is never read.
- Check: e, o, o, l, o, e = 6 vowels.

Q10 · Format specifiers, exactly

Question. `int n = 42; float f = 3.14159;` Write the exact output (use _ for a space) and explain each format specifier.

Statement	Output	Why
`printf("[%5d]", n);`	`[___42]`	width 5, **right**-aligned: 3 spaces + 2 digits

Q10 · Format specifiers, exactly

Question. `int n = 42; float f = 3.14159;` Write the exact output (use _ for a space) and explain each format specifier.

Statement	Output	Why
`printf("[%5d]", n);`	`[___42]`	width 5, **right**-aligned: 3 spaces + 2 digits
`printf("[%-5d]", n);`	`[42___]`	- means **left**-aligned: padding goes after

Q10 · Format specifiers, exactly

Question. `int n = 42; float f = 3.14159;` Write the exact output (use _ for a space) and explain each format specifier.

Statement	Output	Why
`printf("[%5d]", n);`	`[___42]`	width 5, **right**-aligned: 3 spaces + 2 digits
`printf("[%-5d]", n);`	`[42___]`	- means **left**-aligned: padding goes after
`printf("[%.2f]", f);`	`[3.14]`	2 digits after the point (rounded)

Q10 · Format specifiers, exactly

Question. `int n = 42; float f = 3.14159;` Write the exact output (use _ for a space) and explain each format specifier.

Statement	Output	Why
`printf("[%5d]", n);`	`[___42]`	width 5, **right**-aligned: 3 spaces + 2 digits
`printf("[%-5d]", n);`	`[42___]`	- means **left**-aligned: padding goes after
`printf("[%.2f]", f);`	`[3.14]`	2 digits after the point (rounded)
`printf("[%8.3f]", f);`	`[___3.142]`	total width 8 *including* the point: `3.142` is 5 chars, + 3 spaces. Rounded: `3.14159` → `3.142`

Q10 · Format specifiers, exactly

Question. `int n = 42; float f = 3.14159;` Write the exact output (use _ for a space) and explain each format specifier.

Statement	Output	Why
`printf("[%5d]", n);`	`[___42]`	width 5, **right**-aligned: 3 spaces + 2 digits
`printf("[%-5d]", n);`	`[42___]`	- means **left**-aligned: padding goes after
`printf("[%.2f]", f);`	`[3.14]`	2 digits after the point (rounded)
`printf("[%8.3f]", f);`	`[___3.142]`	total width 8 *including* the point: `3.142` is 5 chars, + 3 spaces. Rounded: `3.14159` → `3.142`
`printf("[%e]", f);`	`[3.141590e+00]`	scientific notation, 6 digits after the point by default: 3.14159×10^{0}

TIP `%8.3f` = field width 8, counting the decimal point; `%-5d` = left-aligned.

Q4 · Basic data types

Question. What are the basic data types in C? Give the typical size and range of each. Describe how `short`, `long`, `signed`, `unsigned` change these ranges, with one example each.

Type	Typical size	Range	Used for
`char`	1 byte	-128 to 127	one character / small integer

Q4 · Basic data types

Question. What are the basic data types in C? Give the typical size and range of each. Describe how `short`, `long`, `signed`, `unsigned` change these ranges, with one example each.

Type	Typical size	Range	Used for
`char`	1 byte	-128 to 127	one character / small integer
`int`	4 bytes	$-2\,147\,483\,648$ to $2\,147\,483\,647$	whole numbers

Q4 · Basic data types

Question. What are the basic data types in C? Give the typical size and range of each. Describe how `short`, `long`, `signed`, `unsigned` change these ranges, with one example each.

Type	Typical size	Range	Used for
`char`	1 byte	-128 to 127	one character / small integer
`int`	4 bytes	$-2\,147\,483\,648$ to $2\,147\,483\,647$	whole numbers
`float`	4 bytes	about $\pm 3.4 \times 10^{38}$, $\approx$ 6–7 significant digits	real numbers

Q4 · Basic data types

Question. What are the basic data types in C? Give the typical size and range of each. Describe how `short`, `long`, `signed`, `unsigned` change these ranges, with one example each.

Type	Typical size	Range	Used for
`char`	1 byte	-128 to 127	one character / small integer
`int`	4 bytes	$-2\,147\,483\,648$ to $2\,147\,483\,647$	whole numbers
`float`	4 bytes	about $\pm 3.4 \times 10^{38}$, $\approx$ 6–7 significant digits	real numbers
`double`	8 bytes	about $\pm 1.7 \times 10^{308}$, $\approx$ 15–16 significant digits	precise real numbers

Q4 · Basic data types

Question. What are the basic data types in C? Give the typical size and range of each. Describe how `short`, `long`, `signed`, `unsigned` change these ranges, with one example each.

Type	Typical size	Range	Used for
`char`	1 byte	-128 to 127	one character / small integer
`int`	4 bytes	$-2\,147\,483\,648$ to $2\,147\,483\,647$	whole numbers
`float`	4 bytes	about $\pm 3.4 \times 10^{38}$, $\approx$ 6–7 significant digits	real numbers
`double`	8 bytes	about $\pm 1.7 \times 10^{308}$, $\approx$ 15–16 significant digits	precise real numbers
`void`	—	no value	"returns nothing", `main(void)`

"Typical" matters: the C standard only guarantees *minimum* ranges (e.g. `int` at least 16 bits). Check with `sizeof` on your machine.

Q4 · Qualifiers: `short`, `long`, `signed`, `unsigned`

- **`short`**: fewer bytes, smaller range
  `short int s = 30000;` (2 bytes: -32768 to 32767)

Q4 · Qualifiers: `short`, `long`, `signed`, `unsigned`

- **`short`**: fewer bytes, smaller range
  `short int s = 30000;` (2 bytes: -32768 to 32767)
- **`long`**: at least as big as `int`, often bigger
  `long long pop = 8000000000LL;` (8 bytes)

Q4 · Qualifiers: `short`, `long`, `signed`, `unsigned`

- **`short`**: fewer bytes, smaller range
  `short int s = 30000;` (2 bytes: -32768 to 32767)
- **`long`**: at least as big as `int`, often bigger
  `long long pop = 8000000000LL;` (8 bytes)
- **`signed`** (the default for `int`): half the range is negative
  `signed char t = -40;` (-128 to 127)

Q4 · Qualifiers: `short`, `long`, `signed`, `unsigned`

- **`short`**: fewer bytes, smaller range
  `short int s = 30000;` (2 bytes: -32768 to 32767)
- **`long`**: at least as big as `int`, often bigger
  `long long pop = 8000000000LL;` (8 bytes)
- **`signed`** (the default for `int`): half the range is negative
  `signed char t = -40;` (-128 to 127)
- **`unsigned`**: no negatives, so the positive range **doubles**
  `unsigned char px = 255;` (0 to 255)
  `unsigned int u = 4000000000u;` (0 to $4\,294\,967\,295$)

Q4 · Qualifiers: `short`, `long`, `signed`, `unsigned`

- **`short`**: fewer bytes, smaller range
  `short int s = 30000;` (2 bytes: -32768 to 32767)
- **`long`**: at least as big as `int`, often bigger
  `long long pop = 8000000000LL;` (8 bytes)
- **`signed`** (the default for `int`): half the range is negative
  `signed char t = -40;` (-128 to 127)
- **`unsigned`**: no negatives, so the positive range **doubles**
  `unsigned char px = 255;` (0 to 255)
  `unsigned int u = 4000000000u;` (0 to $4\,294\,967\,295$)

Real sizes on Windows (MSYS2 gcc), from `q04_sizes.c`:

```
char      1 byte   -128 .. 127
short     2 bytes  -32768 .. 32767
int       4 bytes  -2147483648 .. 2147483647
unsigned  4 bytes  0 .. 4294967295
long      4 bytes  -2147483648 .. 2147483647
long long 8 bytes  -9223372036854775808 ..
                    9223372036854775807
float     4 bytes  up to 3.402823e+38 (6 digits)
double    8 bytes  up to 1.797693e+308 (15 digits)
```

Note: `long` is 4 bytes on Windows but **8 bytes** on 64-bit Linux/macOS.

Q6 · Declaring vs. initializing; garbage values

Question. What is declaring a variable and initializing a variable? What is a garbage value? Explain with an example why a variable should be initialized before it is used in a calculation.

Q6 · Declaring vs. initializing; garbage values

Question. What is declaring a variable and initializing a variable? What is a garbage value? Explain with an example why a variable should be initialized before it is used in a calculation.

- **Declare**: give a name and type; memory is reserved.

```
int count;
```

Q6 · Declaring vs. initializing; garbage values

Question. What is declaring a variable and initializing a variable? What is a garbage value? Explain with an example why a variable should be initialized before it is used in a calculation.

- **Declare**: give a name and type; memory is reserved.
  ```
  int count;
  ```
- **Initialize**: give it a first value *at the moment it's declared*.
  ```
  int total = 0;
  ```

Q6 · Declaring vs. initializing; garbage values

Question. What is declaring a variable and initializing a variable? What is a garbage value? Explain with an example why a variable should be initialized before it is used in a calculation.

- **Declare**: give a name and type; memory is reserved.
  `int count;`
- **Initialize**: give it a first value *at the moment it's declared*.
  `int total = 0;`
- **Garbage value**: an uninitialized local variable holds whatever bits were already in that memory. Unpredictable, and different from run to run.

Q6 · Declaring vs. initializing; garbage values

Question. What is declaring a variable and initializing a variable? What is a garbage value? Explain with an example why a variable should be initialized before it is used in a calculation.

- **Declare**: give a name and type; memory is reserved.
  `int count;`
- **Initialize**: give it a first value *at the moment it's declared*.
  `int total = 0;`
- **Garbage value**: an uninitialized local variable holds whatever bits were already in that memory. Unpredictable, and different from run to run.

```c
int count;          /* garbage   */
int total = 0;      /* initialized */
for (int i = 1; i <= 3; i++)
{
    count = count + 1;
    total = total + i;
}
printf("count = %d (expected 3)\n", count);
printf("total = %d (expected 6)\n", total);
```

```
count = 32761 (expected 3)
total = 6 (expected 6)
```

`count` started from garbage, so the sum is meaningless.

TIP Always initialise accumulators (`sum = 0`, `product = 1`), or you will get garbage values.

Q7 · Symbolic constants

Question. What is a symbolic constant? Give two advantages. Write a program using symbolic constants: a shop bill that adds VAT at 7% and gives a 5% discount when the bill is above 2,000 Baht.

Q7 · Symbolic constants

Question. What is a symbolic constant? Give two advantages. Write a program using symbolic constants: a shop bill that adds VAT at 7% and gives a 5% discount when the bill is above 2,000 Baht.

- A **symbolic constant** is a *name* for a fixed value: `#define VAT_RATE 0.07`. The preprocessor replaces every VAT_RATE with `0.07` before compiling. (No = and no `;`!)

Q7 · Symbolic constants

Question. What is a symbolic constant? Give two advantages. Write a program using symbolic constants: a shop bill that adds VAT at 7% and gives a 5% discount when the bill is above 2,000 Baht.

- A **symbolic constant** is a *name* for a fixed value: `#define VAT_RATE 0.07`. The preprocessor replaces every VAT_RATE with `0.07` before compiling. (No = and no `;`!)
- **Advantage 1 – readability**: `bill * VAT_RATE` explains itself; `bill * 0.07` is a "magic number".

Q7 · Symbolic constants

Question. What is a symbolic constant? Give two advantages. Write a program using symbolic constants: a shop bill that adds VAT at 7% and gives a 5% discount when the bill is above 2,000 Baht.

- A **symbolic constant** is a *name* for a fixed value: `#define VAT_RATE 0.07`. The preprocessor replaces every VAT_RATE with `0.07` before compiling. (No = and no `;`!)
- **Advantage 1 – readability**: `bill * VAT_RATE` explains itself; `bill * 0.07` is a "magic number".
- **Advantage 2 – easy to change**: if VAT becomes 10%, edit **one** line, not every place 0.07 appears (and you can't miss one).

Q7 · Symbolic constants

Question. What is a symbolic constant? Give two advantages. Write a program using symbolic constants: a shop bill that adds VAT at 7% and gives a 5% discount when the bill is above 2,000 Baht.

- A **symbolic constant** is a *name* for a fixed value: `#define VAT_RATE 0.07`. The preprocessor replaces every VAT_RATE with `0.07` before compiling. (No = and no `;`!)
- **Advantage 1 – readability**: `bill * VAT_RATE` explains itself; `bill * 0.07` is a "magic number".
- **Advantage 2 – easy to change**: if VAT becomes 10%, edit **one** line, not every place 0.07 appears (and you can't miss one).
- Also: the value can't be changed by accident while the program runs.

Alternative: `const double VAT_RATE = 0.07;`, a typed, read-only variable.

Q7 · Shop bill program

```c
#include <stdio.h>
#define VAT_RATE       0.07
#define DISCOUNT_RATE  0.05
#define DISCOUNT_LIMIT 2000.0
int main(void)
{
    double bill, discount = 0.0, vat, total;
    printf("Enter bill amount (Baht): ");
    scanf("%lf", &bill);
    if (bill > DISCOUNT_LIMIT)
        discount = bill * DISCOUNT_RATE;
    vat   = (bill - discount) * VAT_RATE;
    total = bill - discount + vat;
    printf("Bill     : %10.2f\n", bill);
    printf("Discount : %10.2f\n", discount);
    printf("VAT 7%%   : %10.2f\n", vat);
    printf("Total    : %10.2f\n", total);
    return 0;
}
```

Assumption: discount first, then VAT on the discounted price.

```
Enter bill amount (Baht): 2500
Bill     :    2500.00
Discount :     125.00
VAT 7%   :     166.25
Total    :    2541.25
```

```
Enter bill amount (Baht): 2000
Bill     :    2000.00
Discount :       0.00
VAT 7%   :     140.00
Total    :    2140.00
```

2000 gets no discount: “above” means >, not >=. Note %% prints a single %.

Q11 · Compound interest

Question. $A = P(1 + R/100)^T$. Input P, R and T, then display the final amount and the total interest earned. State which header file is needed and why.

Q11 · Compound interest

Question. $A = P(1 + R/100)^T$. Input P, R and T, then display the final amount and the total interest earned. State which header file is needed and why.

```c
#include <stdio.h>
#include <math.h>          /* for pow() */

int main(void)
{
    double P, R, T, A;

    printf("Principal (P): ");
    scanf("%lf", &P);
    printf("Rate %% per year (R): ");
    scanf("%lf", &R);
    printf("Time in years (T): ");
    scanf("%lf", &T);

    A = P * pow(1 + R / 100, T);

    printf("Final amount   = %.2f\n", A);
    printf("Interest earned = %.2f\n", A - P);
    return 0;
}
```

Q11 · Compound interest

Question. $A = P(1 + R/100)^T$. Input P, R and T, then display the final amount and the total interest earned. State which header file is needed and why.

```c
#include <stdio.h>
#include <math.h>          /* for pow() */

int main(void)
{
    double P, R, T, A;

    printf("Principal (P): ");
    scanf("%lf", &P);
    printf("Rate %% per year (R): ");
    scanf("%lf", &R);
    printf("Time in years (T): ");
    scanf("%lf", &T);

    A = P * pow(1 + R / 100, T);

    printf("Final amount   = %.2f\n", A);
    printf("Interest earned = %.2f\n", A - P);
    return 0;
}
```

```
Principal (P): 10000
Rate % per year (R): 5
Time in years (T): 3
Final amount   = 11576.25
Interest earned = 1576.25
```

Check: $10000 \times 1.05^3 = 11576.25$ ✓

Q11 · Compound interest

Question. $A = P(1 + R/100)^T$. Input P, R and T, then display the final amount and the total interest earned. State which header file is needed and why.

```c
#include <stdio.h>
#include <math.h>          /* for pow() */

int main(void)
{
    double P, R, T, A;

    printf("Principal (P): ");
    scanf("%lf", &P);
    printf("Rate %% per year (R): ");
    scanf("%lf", &R);
    printf("Time in years (T): ");
    scanf("%lf", &T);

    A = P * pow(1 + R / 100, T);

    printf("Final amount   = %.2f\n", A);
    printf("Interest earned = %.2f\n", A - P);
    return 0;
}
```

```
Principal (P): 10000
Rate % per year (R): 5
Time in years (T): 3
Final amount   = 11576.25
Interest earned = 1576.25
```

Check: $10000 \times 1.05^3 = 11576.25$ ✓

Header: `<math.h>`

C has no power operator (^ is XOR!). `pow(x, y)` computes x^y and is declared in `<math.h>`. Link with `-lm` on Linux.

Q12 · Final velocity, with a safety check

Question. $v = \sqrt{u^2 + 2as}$. Read u, a, s and compute v. Print an error message when the value under the square root is negative.

Q12 · Final velocity, with a safety check

Question. $v = \sqrt{u^2 + 2as}$. Read u, a, s and compute v. Print an error message when the value under the square root is negative.

```c
#include <stdio.h>
#include <math.h>          /* for sqrt() */

int main(void)
{
    double u, a, s, inside;

    printf("Enter u, a, s: ");
    scanf("%lf %lf %lf", &u, &a, &s);

    inside = u * u + 2 * a * s;
    if (inside < 0)
    {
        printf("Error: u^2 + 2as = %.2f is negative,"
               " no real velocity.\n", inside);
        return 1;
    }
    printf("v = %.2f\n", sqrt(inside));
    return 0;
}
```

Q12 · Final velocity, with a safety check

Question. $v = \sqrt{u^2 + 2as}$. Read u, a, s and compute v. Print an error message when the value under the square root is negative.

```c
#include <stdio.h>
#include <math.h>          /* for sqrt() */

int main(void)
{
    double u, a, s, inside;

    printf("Enter u, a, s: ");
    scanf("%lf %lf %lf", &u, &a, &s);

    inside = u * u + 2 * a * s;
    if (inside < 0)
    {
        printf("Error: u^2 + 2as = %.2f is negative,"
               " no real velocity.\n", inside);
        return 1;
    }
    printf("v = %.2f\n", sqrt(inside));
    return 0;
}
```

```
Enter u, a, s: 10 2 24
v = 14.00
```

$100 + 2 \cdot 2 \cdot 24 = 196,\ \sqrt{196} = 14$ ✓

```
Enter u, a, s: 3 -2 10
Error: u^2 + 2as = -31.00 is negative, no real
  velocity.
```

Q12 · Final velocity, with a safety check

Question. $v = \sqrt{u^2 + 2as}$. Read u, a, s and compute v. Print an error message when the value under the square root is negative.

```c
#include <stdio.h>
#include <math.h>          /* for sqrt() */

int main(void)
{
    double u, a, s, inside;

    printf("Enter u, a, s: ");
    scanf("%lf %lf %lf", &u, &a, &s);

    inside = u * u + 2 * a * s;
    if (inside < 0)
    {
        printf("Error: u^2 + 2as = %.2f is negative,"
               " no real velocity.\n", inside);
        return 1;
    }
    printf("v = %.2f\n", sqrt(inside));
    return 0;
}
```

```
Enter u, a, s: 10 2 24
v = 14.00
```

$100 + 2 \cdot 2 \cdot 24 = 196$, $\sqrt{196} = 14$ ✓

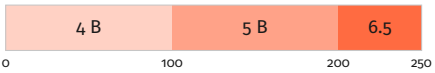
```
Enter u, a, s: 3 -2 10
Error: u^2 + 2as = -31.00 is negative, no real
   velocity.
```

- Compute the inside **once**, test it, *then* call `sqrt`.
- `sqrt` of a negative number gives `nan` (not a number).
- `u * u` is simpler and exact; `pow(u, 2)` also works.

Q13 · Electricity bill: think in slabs

Question. First 100 units at 4 Baht, next 100 units at 5 Baht, every unit above 200 at 6.50 Baht. A fixed service charge of 25 Baht is added. Input a customer ID and units used; display the total bill.

Each slab is charged at its own rate:



Worked example: 250 units

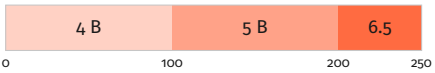
first 100 × 4	400.00

Mistake to avoid: charging *all* units at the highest rate (250×6.50). Only the units *above* 200 get 6.50.

Q13 · Electricity bill: think in slabs

Question. First 100 units at 4 Baht, next 100 units at 5 Baht, every unit above 200 at 6.50 Baht. A fixed service charge of 25 Baht is added. Input a customer ID and units used; display the total bill.

Each slab is charged at its own rate:



Mistake to avoid: charging *all* units at the highest rate (250×6.50). Only the units *above* 200 get 6.50.

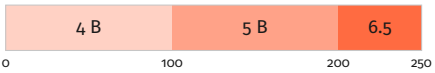
Worked example: 250 units

first 100 $\times$ 4	400.00
next 100 $\times$ 5	500.00

Q13 · Electricity bill: think in slabs

Question. First 100 units at 4 Baht, next 100 units at 5 Baht, every unit above 200 at 6.50 Baht. A fixed service charge of 25 Baht is added. Input a customer ID and units used; display the total bill.

Each slab is charged at its own rate:



Mistake to avoid: charging *all* units at the highest rate (250×6.50). Only the units *above* 200 get 6.50.

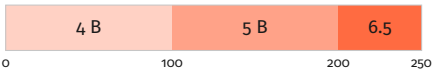
Worked example: 250 units

first 100 $\times$ 4	400.00
next 100 $\times$ 5	500.00
remaining 50 $\times$ 6.50	325.00

Q13 · Electricity bill: think in slabs

Question. First 100 units at 4 Baht, next 100 units at 5 Baht, every unit above 200 at 6.50 Baht. A fixed service charge of 25 Baht is added. Input a customer ID and units used; display the total bill.

Each slab is charged at its own rate:



Mistake to avoid: charging *all* units at the highest rate (250×6.50). Only the units *above* 200 get 6.50.

Worked example: 250 units

first 100 × 4	400.00
next 100 × 5	500.00
remaining 50 × 6.50	325.00
service charge	25.00
Total	**1250.00**

TIP Tiered electricity pricing: each band is charged at its own rate only, not the whole amount at the highest rate.

Q13 · Solution

```c
#include <stdio.h>

int main(void)
{
    int id;
    double units, bill;

    printf("Customer ID: ");
    scanf("%d", &id);
    printf("Units used : ");
    scanf("%lf", &units);

    if (units <= 100)
        bill = units * 4;
    else if (units <= 200)
        bill = 100 * 4 + (units - 100) * 5;
    else
        bill = 100 * 4 + 100 * 5 + (units - 200) * 6.50;

    bill = bill + 25;                       /* service charge */

    printf("Customer %d: total bill = %.2f Baht\n", id, bill);
    return 0;
}
```

```
Customer ID: 1002
Units used : 80
Customer 1002: total bill = 345.00 Baht
```

```
Customer ID: 1003
Units used : 150
Customer 1003: total bill = 675.00 Baht
```

```
Customer ID: 1001
Units used : 250
Customer 1001: total bill = 1250.00 Baht
```

Test **one value in each slab**. Here:

$80 \cdot 4 + 25 = 345,$

$400 + 50 \cdot 5 + 25 = 675.$

Q14 · Courier fees: `switch` + `if`

Question. Use `switch` for the zone (1, 2 or 3) and `if` statements for the weight to calculate the delivery fee. Print an error message for an invalid zone code.

Weight	Zone 1	Zone 2	Zone 3
up to 1 kg	40	60	350
above 1 to 5 kg	70	110	800
above 5 to 10 kg	120	180	1500
above 10 kg	200	300	2500

Q14 · Courier fees: `switch` + `if`

Question. Use `switch` for the zone (1, 2 or 3) and `if` statements for the weight to calculate the delivery fee. Print an error message for an invalid zone code.

Weight	Zone 1	Zone 2	Zone 3
up to 1 kg	40	60	350
above 1 to 5 kg	70	110	800
above 5 to 10 kg	120	180	1500
above 10 kg	200	300	2500

- `switch (zone)` picks the **column**. `default:` catches every invalid zone.

Q14 · Courier fees: `switch` + `if`

Question. Use `switch` for the zone (1, 2 or 3) and `if` statements for the weight to calculate the delivery fee. Print an error message for an invalid zone code.

Weight	Zone 1	Zone 2	Zone 3
up to 1 kg	40	60	350
above 1 to 5 kg	70	110	800
above 5 to 10 kg	120	180	1500
above 10 kg	200	300	2500

- `switch (zone)` picks the **column**. `default:` catches every invalid zone.
- Inside each `case`, an `if / else if` ladder picks the **row**.

Q14 · Courier fees: `switch` + `if`

Question. Use `switch` for the zone (1, 2 or 3) and `if` statements for the weight to calculate the delivery fee. Print an error message for an invalid zone code.

Weight	Zone 1	Zone 2	Zone 3
up to 1 kg	40	60	350
above 1 to 5 kg	70	110	800
above 5 to 10 kg	120	180	1500
above 10 kg	200	300	2500

- `switch (zone)` picks the **column**. `default:` catches every invalid zone.
- Inside each `case`, an `if / else if` ladder picks the **row**.
- Boundaries: "up to 1 kg" ⇒ `w <= 1`; "above 1 to 5" ⇒ `w <= 5` (we already know `w > 1`).

Q14 · Courier fees: `switch` + `if`

Question. Use `switch` for the zone (1, 2 or 3) and `if` statements for the weight to calculate the delivery fee. Print an error message for an invalid zone code.

Weight	Zone 1	Zone 2	Zone 3
up to 1 kg	40	60	350
above 1 to 5 kg	70	110	800
above 5 to 10 kg	120	180	1500
above 10 kg	200	300	2500

- `switch (zone)` picks the **column**. `default:` catches every invalid zone.
- Inside each `case`, an `if / else if` ladder picks the **row**.
- Boundaries: "up to 1 kg" ⇒ `w <= 1`; "above 1 to 5" ⇒ `w <= 5` (we already know `w > 1`).
- Don't forget `break;` after each case!

Q14 · Solution

```c
#include <stdio.h>

int main(void)
{
    int zone;
    double w, fee;

    printf("Zone (1-3): ");
    scanf("%d", &zone);
    printf("Weight (kg): ");
    scanf("%lf", &w);

    switch (zone)
    {
        case 1:
            if (w <= 1)       fee = 40;
            else if (w <= 5)  fee = 70;
            else if (w <= 10) fee = 120;
            else              fee = 200;
            break;
        case 2:
            if (w <= 1)       fee = 60;
            else if (w <= 5)  fee = 110;
            else if (w <= 10) fee = 180;
            else              fee = 300;
            break;
        case 3:
            if (w <= 1)       fee = 350;
            else if (w <= 5)  fee = 800;
            else if (w <= 10) fee = 1500;
            else              fee = 2500;
            break;
        default:
            printf("Error: invalid zone code %d\n", zone);
            return 1;
    }
    printf("Delivery fee = %.0f Baht\n", fee);
    return 0;
}
```

Q14 · Test runs (including the edges)

```
Zone (1-3): 2
Weight (kg): 3.5
Delivery fee = 110 Baht
```

```
Zone (1-3): 3
Weight (kg): 12
Delivery fee = 2500 Baht
```

```
Zone (1-3): 1
Weight (kg): 1
Delivery fee = 40 Baht
```

```
Zone (1-3): 4
Weight (kg): 2
Error: invalid zone code 4
```

- **Exactly 1 kg** is “up to 1 kg”, so the fee is 40, not 70. Always test the boundary values.
- Improvement: check `w <= 0` as an invalid weight too.

TIP Always test the boundary values, e.g. 1, 5, 10 kg. That is where < vs <= mistakes happen most.

Q15 · One function, three ways

Question. Read x and find y: $y = x^2 - 1$ if $x < 5$; $y = 50$ if $x = 5$; $y = 2x + 3$ if $x > 5$. Solve using (a) nested `if`, (b) an else-if ladder, (c) the conditional operator `?:`.

(a) nested if

```
if (x < 5)
    y = x * x - 1;
else
{
    if (x == 5)
        y = 50;
    else
        y = 2 * x + 3;
}
```

Q15 · One function, three ways

Question. Read x and find y: $y = x^2 - 1$ if $x < 5$; $y = 50$ if $x = 5$; $y = 2x + 3$ if $x > 5$. Solve using (a) nested `if`, (b) an else-if ladder, (c) the conditional operator `?:`.

(a) nested if

```
if (x < 5)
    y = x * x - 1;
else
{
    if (x == 5)
        y = 50;
    else
        y = 2 * x + 3;
}
```

(b) else-if ladder

```
if (x < 5)
    y = x * x - 1;
else if (x == 5)
    y = 50;
else
    y = 2 * x + 3;
```

Q15 · One function, three ways

Question. Read x and find y: $y = x^2 - 1$ if $x < 5$; $y = 50$ if $x = 5$; $y = 2x + 3$ if $x > 5$. Solve using (a) nested `if`, (b) an else-if ladder, (c) the conditional operator `? :`.

(a) nested if

```
if (x < 5)
    y = x * x - 1;
else
{
    if (x == 5)
        y = 50;
    else
        y = 2 * x + 3;
}
```

(b) else-if ladder

```
if (x < 5)
    y = x * x - 1;
else if (x == 5)
    y = 50;
else
    y = 2 * x + 3;
```

(c) conditional operator

```
y = (x < 5)  ? x * x - 1
  : (x == 5) ? 50
  :            2 * x + 3;
```

`c ? A : B` = A if c is true, else B.

Q15 · One function, three ways

Question. Read x and find y: $y = x^2 - 1$ if $x < 5$; $y = 50$ if $x = 5$; $y = 2x + 3$ if $x > 5$. Solve using (a) nested `if`, (b) an else-if ladder, (c) the conditional operator `? :`.

(a) nested if

```
if (x < 5)
    y = x * x - 1;
else
{
    if (x == 5)
        y = 50;
    else
        y = 2 * x + 3;
}
```

(b) else-if ladder

```
if (x < 5)
    y = x * x - 1;
else if (x == 5)
    y = 50;
else
    y = 2 * x + 3;
```

(c) conditional operator

```
y = (x < 5)  ? x * x - 1
  : (x == 5) ? 50
  :            2 * x + 3;
```

`c ? A : B` = A if c is true, else B.

```
x = 3
nested if: 8, ladder: 8, ?: 8
x = 5
nested if: 50, ladder: 50, ?: 50
x = 7
nested if: 17, ladder: 17, ?: 17
```

All three agree: $3^2 - 1 = 8$, 50, $2 \cdot 7 + 3 = 17$.
Note `x * x`, not `x^2`, and `==`, not `=`.

Q16 · The dangling `else`

Question. The indentation is misleading. (a) Which `if` owns the `else`? (b) Output for marks=40, att=90 and marks=60, att=70? (c) Rewrite with braces so "`Fail`" prints only when marks < 50.

```
if (marks >= 50)
    if (attendance >= 80)
        printf("Pass");
else
    printf("Fail");
```

Q16 · The dangling `else`

Question. The indentation is misleading. (a) Which `if` owns the `else`? (b) Output for marks=40, att=90 and marks=60, att=70? (c) Rewrite with braces so "`Fail`" prints only when marks < 50.

```
if (marks >= 50)
    if (attendance >= 80)
        printf("Pass");
else
    printf("Fail");
```

(a) An `else` pairs with the **nearest unmatched `if`**, here `if (attendance >= 80)`. Indentation means nothing to the compiler. The code really is:

```
if (marks >= 50) {
    if (attendance >= 80) printf("Pass");
    else                  printf("Fail");
}
```

Q16 · The dangling `else`

Question. The indentation is misleading. (a) Which `if` owns the `else`? (b) Output for marks=40, att=90 and marks=60, att=70? (c) Rewrite with braces so "`Fail`" prints only when marks < 50.

```
if (marks >= 50)
    if (attendance >= 80)
        printf("Pass");
else
    printf("Fail");
```

(a) An `else` pairs with the **nearest unmatched `if`**, here `if (attendance >= 80)`. Indentation means nothing to the compiler. The code really is:

```
if (marks >= 50) {
    if (attendance >= 80) printf("Pass");
    else                  printf("Fail");
}
```

(b)

marks=40, att=90	*(nothing)*	outer `if` is false, and it has no `else`
marks=60, att=70	`Fail`	outer true, inner false → its `else`

Q16 · (c) Fix it with braces

```c
if (marks >= 50)
{
    if (attendance >= 80)
        printf("Pass");
}
else
    printf("Fail");
```

The braces close the inner `if`, so the `else` can only belong to the outer `if (marks >= 50)`.

Real run of both versions (`q16_dangling.c`):

```
original(40, 90): []
original(60, 70): [Fail]
fixed(40, 90):    [Fail]
fixed(60, 70):    []
```

gcc even warns about the original:

```
warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
```

TIP An `else` pairs with the nearest unmatched `if`, regardless of indentation. Always use `{ }` to make it clear.

Q17 · A loop that multiplies

Question. Identify the initialization, test condition and update. Write the output, how many times the body runs, and the value of k after the loop.

```
for (k = 1; k <= 100; k = k * 3)
{
    printf("%d ", k);
}
```

Q17 · A loop that multiplies

Question. Identify the initialization, test condition and update. Write the output, how many times the body runs, and the value of k after the loop.

```
for (k = 1; k <= 100; k = k * 3)
{
    printf("%d ", k);
}
```

initialization	`k = 1` (runs once)
test condition	`k <= 100` (before every pass)
update	`k = k * 3` (after every pass)

Q17 · A loop that multiplies

Question. Identify the initialization, test condition and update. Write the output, how many times the body runs, and the value of k after the loop.

```
for (k = 1; k <= 100; k = k * 3)
{
    printf("%d ", k);
}
```

initialization	`k = 1` (runs once)
test condition	`k <= 100` (before every pass)
update	`k = k * 3` (after every pass)

Trace

pass	k	k <= 100?	prints
1	1	yes	1

Q17 · A loop that multiplies

Question. Identify the initialization, test condition and update. Write the output, how many times the body runs, and the value of k after the loop.

```
for (k = 1; k <= 100; k = k * 3)
{
    printf("%d ", k);
}
```

initialization	`k = 1` (runs once)
test condition	`k <= 100` (before every pass)
update	`k = k * 3` (after every pass)

Trace

pass	k	k <= 100?	prints
1	1	yes	1
2	3	yes	3

Q17 · A loop that multiplies

Question. Identify the initialization, test condition and update. Write the output, how many times the body runs, and the value of k after the loop.

```
for (k = 1; k <= 100; k = k * 3)
{
    printf("%d ", k);
}
```

initialization `k = 1` (runs once)
test condition `k <= 100` (before every pass)
update `k = k * 3` (after every pass)

Trace

pass	k	k <= 100?	prints
1	1	yes	1
2	3	yes	3
3	9	yes	9

Q17 · A loop that multiplies

Question. Identify the initialization, test condition and update. Write the output, how many times the body runs, and the value of k after the loop.

```
for (k = 1; k <= 100; k = k * 3)
{
    printf("%d ", k);
}
```

initialization	`k = 1` (runs once)
test condition	`k <= 100` (before every pass)
update	`k = k * 3` (after every pass)

Trace

pass	k	k <= 100?	prints
1	1	yes	1
2	3	yes	3
3	9	yes	9
4	27	yes	27

Q17 · A loop that multiplies

Question. Identify the initialization, test condition and update. Write the output, how many times the body runs, and the value of k after the loop.

```
for (k = 1; k <= 100; k = k * 3)
{
    printf("%d ", k);
}
```

initialization	`k = 1` (runs once)
test condition	`k <= 100` (before every pass)
update	`k = k * 3` (after every pass)

Trace

pass	k	k <= 100?	prints
1	1	yes	1
2	3	yes	3
3	9	yes	9
4	27	yes	27
5	81	yes	81

Q17 · A loop that multiplies

Question. Identify the initialization, test condition and update. Write the output, how many times the body runs, and the value of k after the loop.

```
for (k = 1; k <= 100; k = k * 3)
{
    printf("%d ", k);
}
```

initialization	`k = 1` (runs once)
test condition	`k <= 100` (before every pass)
update	`k = k * 3` (after every pass)

Trace

pass	k	k <= 100?	prints
1	1	yes	1
2	3	yes	3
3	9	yes	9
4	27	yes	27
5	81	yes	81
—	**243**	no: stop	

Q17 · A loop that multiplies

Question. Identify the initialization, test condition and update. Write the output, how many times the body runs, and the value of k after the loop.

```
for (k = 1; k <= 100; k = k * 3)
{
    printf("%d ", k);
}
```

initialization	`k = 1` (runs once)
test condition	`k <= 100` (before every pass)
update	`k = k * 3` (after every pass)

Trace

pass	k	k <= 100?	prints
1	1	yes	1
2	3	yes	3
3	9	yes	9
4	27	yes	27
5	81	yes	81
—	**243**	no: stop	

```
1 3 9 27 81
body ran 5 times, k after loop = 243
```

Q18 · break stops everything

Question. Predict the output. Explain why the final value of `i` is not 11.

```
sum = 0;
for (i = 1; i <= 10; i++)
{
    sum = sum + i;
    if (sum > 20)
        break;
}
printf("i = %d, sum = %d", i, sum);
```

Q18 · break stops everything

Question. Predict the output. Explain why the final value of `i` is not 11.

```
sum = 0;
for (i = 1; i <= 10; i++)
{
    sum = sum + i;
    if (sum > 20)
        break;
}
printf("i = %d, sum = %d", i, sum);
```

i	sum after adding	sum > 20?
1	1	no

Q18 · break stops everything

Question. Predict the output. Explain why the final value of `i` is not 11.

```
sum = 0;
for (i = 1; i <= 10; i++)
{
    sum = sum + i;
    if (sum > 20)
        break;
}
printf("i = %d, sum = %d", i, sum);
```

i	sum after adding	sum > 20?
1	1	no
2	3	no

Q18 · break stops everything

Question. Predict the output. Explain why the final value of `i` is not 11.

```
sum = 0;
for (i = 1; i <= 10; i++)
{
    sum = sum + i;
    if (sum > 20)
        break;
}
printf("i = %d, sum = %d", i, sum);
```

i	sum after adding	sum > 20?
1	1	no
2	3	no
3	6	no

Q18 · break stops everything

Question. Predict the output. Explain why the final value of `i` is not 11.

```
sum = 0;
for (i = 1; i <= 10; i++)
{
    sum = sum + i;
    if (sum > 20)
        break;
}
printf("i = %d, sum = %d", i, sum);
```

i	sum after adding	sum > 20?
1	1	no
2	3	no
3	6	no
4	10	no

Q18 · break stops everything

Question. Predict the output. Explain why the final value of `i` is not 11.

```
sum = 0;
for (i = 1; i <= 10; i++)
{
    sum = sum + i;
    if (sum > 20)
        break;
}
printf("i = %d, sum = %d", i, sum);
```

i	sum after adding	sum > 20?
1	1	no
2	3	no
3	6	no
4	10	no
5	15	no

Q18 · break stops everything

Question. Predict the output. Explain why the final value of `i` is not 11.

```
sum = 0;
for (i = 1; i <= 10; i++)
{
    sum = sum + i;
    if (sum > 20)
        break;
}
printf("i = %d, sum = %d", i, sum);
```

i	sum after adding	sum > 20?
1	1	no
2	3	no
3	6	no
4	10	no
5	15	no
6	21	**yes** → break

Q18 · break stops everything

Question. Predict the output. Explain why the final value of `i` is not 11.

```
sum = 0;
for (i = 1; i <= 10; i++)
{
    sum = sum + i;
    if (sum > 20)
        break;
}
printf("i = %d, sum = %d", i, sum);
```

i	sum after adding	sum > 20?
1	1	no
2	3	no
3	6	no
4	10	no
5	15	no
6	21	**yes** → break

```
i = 6, sum = 21
```

Why not 11? `i` reaches 11 only if `i <= 10` becomes false. `break` exits **immediately**, skipping `i++`, so `i` stays 6.

Q19 · `continue` skips one pass

Question. Write the output. Explain how `continue` differs from `break` in this loop.

```
for (i = 1; i <= 15; i++)
{
    if (i % 2 == 0 || i % 5 == 0)
        continue;
    printf("%d ", i);
}
```

Q19 · `continue` skips one pass

Question. Write the output. Explain how `continue` differs from `break` in this loop.

```
for (i = 1; i <= 15; i++)
{
    if (i % 2 == 0 || i % 5 == 0)
        continue;
    printf("%d ", i);
}
```

Skip `i` if it is **even** or a **multiple of 5**:



Q19 · `continue` skips one pass

Question. Write the output. Explain how `continue` differs from `break` in this loop.

```c
for (i = 1; i <= 15; i++)
{
    if (i % 2 == 0 || i % 5 == 0)
        continue;
    printf("%d ", i);
}
```

Skip `i` if it is **even** or a **multiple of 5**:




```
1 3 7 9 11 13
```

Q19 · `continue` skips one pass

Question. Write the output. Explain how `continue` differs from `break` in this loop.

```
for (i = 1; i <= 15; i++)
{
    if (i % 2 == 0 || i % 5 == 0)
        continue;
    printf("%d ", i);
}
```

Skip `i` if it is **even** or a **multiple of 5**:




```
1 3 7 9 11 13
```

- `continue`: skip the **rest of this pass**, go to `i++`, and keep looping.
- `break`: leave the loop **completely**. With `break` here, the loop would stop at `i = 2` and print only `1`.

Q20 · Counting nested-loop executions (part 1)

Question. How many times will `printf()` execute?

```
for (i = 0; i < 4; i++)
{
    for (j = i; j < 6; j++)
    {
        printf("#");
    }
}
```

i	j values	**times**

The inner loop runs for $j = i, i+1, \ldots, 5$, which is $6 - i$ values.

Q20 · Counting nested-loop executions (part 1)

Question. How many times will `printf()` execute?

```
for (i = 0; i < 4; i++)
{
    for (j = i; j < 6; j++)
    {
        printf("#");
    }
}
```

i	j values	**times**
0	0 1 2 3 4 5	6

The inner loop runs for $j = i, i+1, \ldots, 5$, which is $6 - i$ values.

Q20 · Counting nested-loop executions (part 1)

Question. How many times will `printf()` execute?

```
for (i = 0; i < 4; i++)
{
    for (j = i; j < 6; j++)
    {
        printf("#");
    }
}
```

i	j values	**times**
0	0 1 2 3 4 5	6
1	1 2 3 4 5	5

The inner loop runs for $j = i, i+1, \ldots, 5$, which is $6 - i$ values.

Q20 · Counting nested-loop executions (part 1)

Question. How many times will `printf()` execute?

```
for (i = 0; i < 4; i++)
{
    for (j = i; j < 6; j++)
    {
        printf("#");
    }
}
```

The inner loop runs for $j = i, i+1, \ldots, 5$, which is $6 - i$ values.

i	j values	**times**
0	0 1 2 3 4 5	6
1	1 2 3 4 5	5
2	2 3 4 5	4

Q20 · Counting nested-loop executions (part 1)

Question. How many times will `printf()` execute?

```
for (i = 0; i < 4; i++)
{
    for (j = i; j < 6; j++)
    {
        printf("#");
    }
}
```

The inner loop runs for $j = i, i+1, \ldots, 5$, which is $6 - i$ values.

i	j values	**times**
0	0 1 2 3 4 5	6
1	1 2 3 4 5	5
2	2 3 4 5	4
3	3 4 5	3
	total	**18**

$6 + 5 + 4 + 3 = 18$. Verified by counting in a real run: `part 1: 18`.

Q20 · Counting nested-loop executions (part 2)

Question. Determine the total number of times `printf()` is executed. Show your working for each `i`.

```
for (i = 1; i <= 6; i++)
{
    for (j = 1; j <= i; j = j + 2)
    {
        printf("*");
    }
}
```

i	j values	**times**

`j` jumps by **2**: $1, 3, 5, \ldots$ while $j \leq i$. That's $\lceil i/2 \rceil$ values.

Q20 · Counting nested-loop executions (part 2)

Question. Determine the total number of times `printf()` is executed. Show your working for each `i`.

```
for (i = 1; i <= 6; i++)
{
    for (j = 1; j <= i; j = j + 2)
    {
        printf("*");
    }
}
```

i	j values	**times**
1	1	1

`j` jumps by **2**: $1, 3, 5, \ldots$ while $j \leq i$. That's $\lceil i/2 \rceil$ values.

Q20 · Counting nested-loop executions (part 2)

Question. Determine the total number of times `printf()` is executed. Show your working for each `i`.

```
for (i = 1; i <= 6; i++)
{
    for (j = 1; j <= i; j = j + 2)
    {
        printf("*");
    }
}
```

i	j values	**times**
1	1	1
2	1	1

`j` jumps by **2**: $1, 3, 5, \ldots$ while $j \leq i$. That's $\lceil i/2 \rceil$ values.

Q20 · Counting nested-loop executions (part 2)

Question. Determine the total number of times `printf()` is executed. Show your working for each `i`.

```
for (i = 1; i <= 6; i++)
{
    for (j = 1; j <= i; j = j + 2)
    {
        printf("*");
    }
}
```

`j` jumps by **2**: $1, 3, 5, \ldots$ while $j \leq i$. That's $\lceil i/2 \rceil$ values.

`i`	`j` values	**times**
1	1	1
2	1	1
3	1 3	2

Q20 · Counting nested-loop executions (part 2)

Question. Determine the total number of times `printf()` is executed. Show your working for each `i`.

```
for (i = 1; i <= 6; i++)
{
    for (j = 1; j <= i; j = j + 2)
    {
        printf("*");
    }
}
```

`j` jumps by **2**: $1, 3, 5, \ldots$ while $j \leq i$. That's $\lceil i/2 \rceil$ values.

`i`	`j` values	**times**
1	1	1
2	1	1
3	1 3	2
4	1 3	2

Q20 · Counting nested-loop executions (part 2)

Question. Determine the total number of times `printf()` is executed. Show your working for each `i`.

```
for (i = 1; i <= 6; i++)
{
    for (j = 1; j <= i; j = j + 2)
    {
        printf("*");
    }
}
```

`j` jumps by **2**: $1, 3, 5, \ldots$ while $j \leq i$. That's $\lceil i/2 \rceil$ values.

i	j values	**times**
1	1	1
2	1	1
3	1 3	2
4	1 3	2
5	1 3 5	3

Q20 · Counting nested-loop executions (part 2)

Question. Determine the total number of times `printf()` is executed. Show your working for each `i`.

```
for (i = 1; i <= 6; i++)
{
    for (j = 1; j <= i; j = j + 2)
    {
        printf("*");
    }
}
```

`j` jumps by **2**: $1, 3, 5, \ldots$ while $j \le i$. That's $\lceil i/2 \rceil$ values.

i	j values	**times**
1	1	1
2	1	1
3	1 3	2
4	1 3	2
5	1 3 5	3
6	1 3 5	3
	total	**12**

$1 + 1 + 2 + 2 + 3 + 3 = 12$ (real run: `part 2: 12`).

Q21 · Trace a nested loop

Question. Trace and write the complete output. How does the number of inner-loop executions change after each outer iteration?

```
for (i = 1; i <= 4; i++)
{
    for (j = 1; j <= 5 - i; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}
```

i	5 - i	inner runs	line printed

Q21 · Trace a nested loop

Question. Trace and write the complete output. How does the number of inner-loop executions change after each outer iteration?

```
for (i = 1; i <= 4; i++)
{
    for (j = 1; j <= 5 - i; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}
```

i	5 - i	inner runs	line printed
1	4	4	1 2 3 4

Q21 · Trace a nested loop

Question. Trace and write the complete output. How does the number of inner-loop executions change after each outer iteration?

```
for (i = 1; i <= 4; i++)
{
    for (j = 1; j <= 5 - i; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}
```

i	5 - i	inner runs	line printed
1	4	4	1 2 3 4
2	3	3	1 2 3

Q21 · Trace a nested loop

Question. Trace and write the complete output. How does the number of inner-loop executions change after each outer iteration?

```
for (i = 1; i <= 4; i++)
{
    for (j = 1; j <= 5 - i; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}
```

i	5 - i	inner runs	line printed
1	4	4	1 2 3 4
2	3	3	1 2 3
3	2	2	1 2

Q21 · Trace a nested loop

Question. Trace and write the complete output. How does the number of inner-loop executions change after each outer iteration?

```
for (i = 1; i <= 4; i++)
{
    for (j = 1; j <= 5 - i; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}
```

i	5 - i	inner runs	line printed
1	4	4	1 2 3 4
2	3	3	1 2 3
3	2	2	1 2
4	1	1	1

Q21 · Trace a nested loop

Question. Trace and write the complete output. How does the number of inner-loop executions change after each outer iteration?

```
for (i = 1; i <= 4; i++)
{
    for (j = 1; j <= 5 - i; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}
```

i	5 - i	inner runs	line printed
1	4	4	1 2 3 4
2	3	3	1 2 3
3	2	2	1 2
4	1	1	1

```
1 2 3 4
1 2 3
1 2
1
```

Inner loop: **one run fewer** per outer pass (4, 3, 2, 1), because its limit `5 - i` shrinks as `i` grows.

Q22 · `while` vs. `do-while`

Question. How many times is `*` printed in each, and what is the final `k`? What if `k` started at 3?

```c
int k = 0;
while (k > 0)
{
    printf("*");
    k--;
}
```

```c
int k = 0;
do
{
    printf("*");
    k--;
} while (k > 0);
```

Q22 · `while` vs. `do-while`

Question. How many times is `*` printed in each, and what is the final `k`? What if `k` started at 3?

```c
int k = 0;
while (k > 0)
{
    printf("*");
    k--;
}
```

```c
int k = 0;
do
{
    printf("*");
    k--;
} while (k > 0);
```

	start `k = 0`		start `k = 3`	
	`*` printed	final `k`	`*` printed	final `k`
`while`	0	0	3	0

Q22 · `while` vs. `do-while`

Question. How many times is `*` printed in each, and what is the final `k`? What if `k` started at 3?

```c
int k = 0;
while (k > 0)
{
    printf("*");
    k--;
}
```

```c
int k = 0;
do
{
    printf("*");
    k--;
} while (k > 0);
```

	start `k = 0`		start `k = 3`	
	`*` printed	final `k`	`*` printed	final `k`
`while`	0	0	3	0
`do-while`	**1**	−1	3	0

```
start k = 0
  while   : [] k = 0
  do-while: [*] k = -1
```

```
start k = 3
  while   : [***] k = 0
  do-while: [***] k = 0
```

`while` tests **before** the body, so it may run **0** times. `do-while` tests **after**, so it runs **at least once**. With `k = 3` they behave the same.

Q23 · Prefix and postfix `--`

Question. Find `a`, `b`, `c`. Then explain why an expression such as `a++ + a++` should never be used.

```
int a = 10;
int b = a--;
int c = --a;
```

`a--` (postfix)	**use** the old value, **then** decrease
`--a` (prefix)	**decrease** first, then use the new value

after line	a	b	c
`int a = 10;`	10	–	–

Q23 · Prefix and postfix `--`

Question. Find `a`, `b`, `c`. Then explain why an expression such as `a++ + a++` should never be used.

```
int a = 10;
int b = a--;
int c = --a;
```

`a--` (postfix) **use** the old value, **then** decrease

`--a` (prefix) **decrease** first, then use the new value

after line	a	b	c
`int a = 10;`	10	–	–
`int b = a--;`	9	**10**	–

Q23 · Prefix and postfix `--`

Question. Find `a`, `b`, `c`. Then explain why an expression such as `a++ + a++` should never be used.

```
int a = 10;
int b = a--;
int c = --a;
```

`a--` (postfix) **use** the old value, **then** decrease

`--a` (prefix) **decrease** first, then use the new value

after line	a	b	c
`int a = 10;`	10	–	–
`int b = a--;`	9	**10**	–
`int c = --a;`	**8**	10	**8**

```
a = 8, b = 10, c = 8
```

Q23 · Prefix and postfix `--`

Question. Find `a`, `b`, `c`. Then explain why an expression such as `a++ + a++` should never be used.

```
int a = 10;
int b = a--;
int c = --a;
```

`a--` (postfix) **use** the old value, **then** decrease

`--a` (prefix) **decrease** first, then use the new value

after line	a	b	c
`int a = 10;`	10	–	–
`int b = a--;`	9	**10**	–
`int c = --a;`	**8**	10	**8**

```
a = 8, b = 10, c = 8
```

`b = a--`: b gets 10, *then* a becomes 9.

`c = --a`: a becomes 8 *first*, then c gets 8.

Q23 · Why a++ + a++ is forbidden

```c
int a = 5;
int x = a++ + a++;   /* ??? */
printf("x = %d, a = %d\n", x, a);
```

```
warning: operation on 'a' may be undefined
  [-Wsequence-point]
x = 11, a = 7
```

Our gcc printed `x = 11`. Is that “correct”? There **is no** correct answer.

TIP Never modify the same variable more than once in a single expression. The result is unpredictable (undefined behaviour).

Q23 · Why a++ + a++ is forbidden

```c
int a = 5;
int x = a++ + a++;   /* ??? */
printf("x = %d, a = %d\n", x, a);
```

```
warning: operation on 'a' may be undefined
  [-Wsequence-point]
x = 11, a = 7
```

Our gcc printed `x = 11`. Is that “correct”? There **is no** correct answer.

- a is **modified twice** in one expression, with no *sequence point* in between.

TIP Never modify the same variable more than once in a single expression. The result is unpredictable (undefined behaviour).

Q23 · Why a++ + a++ is forbidden

```c
int a = 5;
int x = a++ + a++;   /* ??? */
printf("x = %d, a = %d\n", x, a);
```

```
warning: operation on 'a' may be undefined
  [-Wsequence-point]
x = 11, a = 7
```

Our gcc printed `x = 11`. Is that “correct”? There **is no** correct answer.

- a is **modified twice** in one expression, with no *sequence point* in between.
- C doesn’t say *when* each `++` takes effect, or which side of `+` is evaluated first.

TIP Never modify the same variable more than once in a single expression. The result is unpredictable (undefined behaviour).

Q23 · Why a++ + a++ is forbidden

```c
int a = 5;
int x = a++ + a++;   /* ??? */
printf("x = %d, a = %d\n", x, a);
```

```
warning: operation on 'a' may be undefined
  [-Wsequence-point]
x = 11, a = 7
```

Our gcc printed `x = 11`. Is that “correct”? There **is no** correct answer.

- a is **modified twice** in one expression, with no *sequence point* in between.
- C doesn’t say *when* each `++` takes effect, or which side of `+` is evaluated first.
- So it is **undefined behaviour**: other compilers or optimisation levels may give 10, 11, 12..., or anything else.

TIP Never modify the same variable more than once in a single expression. The result is unpredictable (undefined behaviour).

Q23 · Why a++ + a++ is forbidden

```c
int a = 5;
int x = a++ + a++;   /* ??? */
printf("x = %d, a = %d\n", x, a);
```

```
warning: operation on 'a' may be undefined
  [-Wsequence-point]
x = 11, a = 7
```

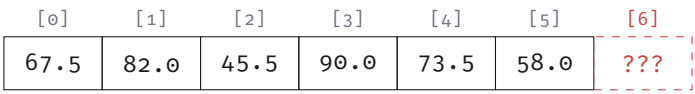
Our gcc printed `x = 11`. Is that “correct”? There **is no** correct answer.

- a is **modified twice** in one expression, with no *sequence point* in between.
- C doesn’t say *when* each `++` takes effect, or which side of `+` is evaluated first.
- So it is **undefined behaviour**: other compilers or optimisation levels may give 10, 11, 12..., or anything else.
- **Rule**: modify a variable at most once per expression. Split it into separate statements.

TIP Never modify the same variable more than once in a single expression. The result is unpredictable (undefined behaviour).

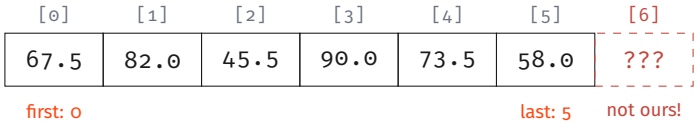
Q24 · Array statistics: plan

Question. `float marks[6] = {67.5, 82.0, 45.5, 90.0, 73.5, 58.0};` Find the highest and lowest mark, the average, and how many marks are above the average. State the first and last valid indices, and what may happen if the program accesses `marks[6]`.



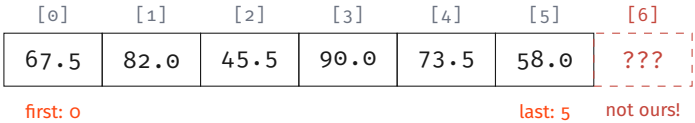
Q24 · Array statistics: plan

Question. `float marks[6] = {67.5, 82.0, 45.5, 90.0, 73.5, 58.0};` Find the highest and lowest mark, the average, and how many marks are above the average. State the first and last valid indices, and what may happen if the program accesses `marks[6]`.



Q24 · Array statistics: plan

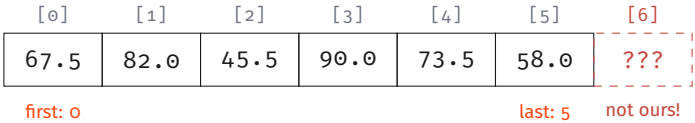
Question. `float marks[6] = {67.5, 82.0, 45.5, 90.0, 73.5, 58.0};` Find the highest and lowest mark, the average, and how many marks are above the average. State the first and last valid indices, and what may happen if the program accesses `marks[6]`.



1. Start with `high = low = marks[0]`, `sum = 0`.
2. One loop `i = 0 .. 5`: update `high` / `low`, add to `sum`.

Q24 · Array statistics: plan

Question. `float marks[6] = {67.5, 82.0, 45.5, 90.0, 73.5, 58.0};` Find the highest and lowest mark, the average, and how many marks are above the average. State the first and last valid indices, and what may happen if the program accesses `marks[6]`.



1. Start with `high = low = marks[0]`, `sum = 0`.
2. One loop `i = 0 .. 5`: update `high` / `low`, add to `sum`.
3. `avg = sum / 6`. A **second** loop counts `marks[i] > avg` (we need the average *before* we can compare).

Q24 · Solution

```c
#include <stdio.h>
#define N 6
int main(void)
{
    float marks[N] = {67.5, 82.0, 45.5, 90.0, 73.5, 58.0};
    float high = marks[0], low = marks[0], sum = 0, avg;
    int i, above = 0;
    for (i = 0; i < N; i++)
    {
        if (marks[i] > high) high = marks[i];
        if (marks[i] < low)  low  = marks[i];
        sum += marks[i];
    }
    avg = sum / N;
    for (i = 0; i < N; i++)
        if (marks[i] > avg)
            above++;
    printf("Highest = %.1f\n", high);
    printf("Lowest  = %.1f\n", low);
    printf("Average = %.2f\n", avg);
    printf("Above average: %d\n", above);
    return 0;
}
```

```
Highest = 90.0
Lowest  = 45.5
Average = 69.42
Above average: 3
```

sum $= 416.5$, avg $= 416.5/6 = 69.4167$

above it: 82.0, 90.0, 73.5 $\Rightarrow$ **3**

Q24 · Solution

```c
#include <stdio.h>
#define N 6
int main(void)
{
    float marks[N] = {67.5, 82.0, 45.5, 90.0, 73.5, 58.0};
    float high = marks[0], low = marks[0], sum = 0, avg;
    int i, above = 0;
    for (i = 0; i < N; i++)
    {
        if (marks[i] > high) high = marks[i];
        if (marks[i] < low)  low  = marks[i];
        sum += marks[i];
    }
    avg = sum / N;
    for (i = 0; i < N; i++)
        if (marks[i] > avg)
            above++;
    printf("Highest = %.1f\n", high);
    printf("Lowest  = %.1f\n", low);
    printf("Average = %.2f\n", avg);
    printf("Above average: %d\n", above);
    return 0;
}
```

```
Highest = 90.0
Lowest  = 45.5
Average = 69.42
Above average: 3
```

sum $= 416.5$, avg $= 416.5/6 = 69.4167$

above it: 82.0, 90.0, 73.5 $\Rightarrow$ **3**

`marks[6]`

Valid indices: **0 to 5**. `marks[6]` is past the end. C does **no bounds checking**: it compiles, but you read garbage, or a write corrupts other variables or crashes (undefined behaviour).

Q25 · The series $\sum 1/[n(n+1)]$

Question. $S = \sum_{n=1}^{\infty} \frac{1}{n(n+1)}$ is evaluated with the loop below. (a) Why do the terms approach zero? (b) Show $S \to 1$. (c) Why `1.0` and not `1`? (d) How does the number of iterations grow with $1/\varepsilon$? Compare with terms like $1/2^n$.

```
n = 1;
sum = 0;
term = 1.0 / (n * (n + 1));
while (term > epsilon)
{
    sum = sum + term;
    n++;
    term = 1.0 / (n * (n + 1));
}
```

(a) Why the terms go to zero

Q25 · The series $\sum 1/[n(n+1)]$

Question. $S = \sum_{n=1}^{\infty} \frac{1}{n(n+1)}$ is evaluated with the loop below. (a) Why do the terms approach zero? (b) Show $S \to 1$. (c) Why `1.0` and not `1`? (d) How does the number of iterations grow with $1/\varepsilon$? Compare with terms like $1/2^n$.

```
n = 1;
sum = 0;
term = 1.0 / (n * (n + 1));
while (term > epsilon)
{
    sum = sum + term;
    n++;
    term = 1.0 / (n * (n + 1));
}
```

(a) Why the terms go to zero

n	$n(n+1)$	term
1	2	0.5
2	6	0.1667
3	12	0.0833
10	110	0.0091
100	10100	0.000099

Q25 · The series $\sum 1/[n(n+1)]$

Question. $S = \sum_{n=1}^{\infty} \frac{1}{n(n+1)}$ is evaluated with the loop below. (a) Why do the terms approach zero? (b) Show $S \to 1$. (c) Why `1.0` and not `1`? (d) How does the number of iterations grow with $1/\varepsilon$? Compare with terms like $1/2^n$.

```
n = 1;
sum = 0;
term = 1.0 / (n * (n + 1));
while (term > epsilon)
{
    sum = sum + term;
    n++;
    term = 1.0 / (n * (n + 1));
}
```

(a) Why the terms go to zero

n	$n(n+1)$	term
1	2	0.5
2	6	0.1667
3	12	0.0833
10	110	0.0091
100	10100	0.000099

The denominator $n(n+1)$ grows without limit (roughly like n^2), while the numerator stays 1. So $\frac{1}{n(n+1)} \to 0$ as $n \to \infty$. The loop therefore stops eventually, for any $\varepsilon > 0$.

Q25 · (b) Why the sum approaches 1: telescoping

Split each term (partial fractions):

$$\frac{1}{n(n+1)} = \frac{1}{n} - \frac{1}{n+1} \qquad \text{check: } \frac{(n+1)-n}{n(n+1)} = \frac{1}{n(n+1)} \checkmark$$

Q25 · (b) Why the sum approaches 1: telescoping

Split each term (partial fractions):

$$\frac{1}{n(n+1)} = \frac{1}{n} - \frac{1}{n+1} \qquad \text{check: } \frac{(n+1)-n}{n(n+1)} = \frac{1}{n(n+1)} \checkmark$$

Add the first N terms: almost everything cancels.

$$S_N = \left(1 - \tfrac{1}{2}\right) + \left(\tfrac{1}{2} - \tfrac{1}{3}\right) + \left(\tfrac{1}{3} - \tfrac{1}{4}\right) + \cdots + \left(\tfrac{1}{N} - \tfrac{1}{N+1}\right)$$

Q25 · (b) Why the sum approaches 1: telescoping

Split each term (partial fractions):

$$\frac{1}{n(n+1)} = \frac{1}{n} - \frac{1}{n+1} \qquad \text{check: } \frac{(n+1)-n}{n(n+1)} = \frac{1}{n(n+1)} \checkmark$$

Add the first N terms: almost everything cancels.

$$S_N = \left(1 - \tfrac{1}{2}\right) + \left(\tfrac{1}{2} - \tfrac{1}{3}\right) + \left(\tfrac{1}{3} - \tfrac{1}{4}\right) + \cdots + \left(\tfrac{1}{N} - \tfrac{1}{N+1}\right)$$

$$S_N = 1 - \frac{1}{N+1} \qquad \Longrightarrow \qquad \lim_{N\to\infty} S_N = 1 - 0 = \boxed{1}$$

Q25 · (b) Why the sum approaches 1: telescoping

Split each term (partial fractions):

$$\frac{1}{n(n+1)} = \frac{1}{n} - \frac{1}{n+1} \qquad \text{check: } \frac{(n+1)-n}{n(n+1)} = \frac{1}{n(n+1)} \checkmark$$

Add the first N terms: almost everything cancels.

$$S_N = \left(1 - \tfrac{1}{2}\right) + \left(\tfrac{1}{2} - \tfrac{1}{3}\right) + \left(\tfrac{1}{3} - \tfrac{1}{4}\right) + \cdots + \left(\tfrac{1}{N} - \tfrac{1}{N+1}\right)$$

$$S_N = 1 - \frac{1}{N+1} \qquad \Longrightarrow \qquad \lim_{N\to\infty} S_N = 1 - 0 = \boxed{1}$$

Check with the program: after $N = 999$ terms, $S_{999} = 1 - \frac{1}{1000} = 0.999$, and the real run prints `sum = 0.999000`. ✓

TIP Telescoping = the middle terms all cancel out, leaving only the first and last.

Q25 · (c) Why `1.0` and not `1`?

`n * (n + 1)` is an `int`. With `1` on top, it's **integer division**: the fraction is thrown away.

`1 / (1 * 2)` $= 1/2 =$ **0**

`1.0 / (1 * 2)` $= 1.0/2 =$ **0.5**

```
1 / (n*(n+1)) with ints = 0,  1.0 / (n*(n+1)) = 0.50
```

Q25 · (c) Why `1.0` and not `1`?

`n * (n + 1)` is an `int`. With `1` on top, it's **integer division**: the fraction is thrown away.

`1 / (1 * 2)` $= 1/2 =$ **0**
`1.0 / (1 * 2)` $= 1.0/2 =$ **0.5**

```
1 / (n*(n+1)) with ints = 0,  1.0 / (n*(n+1)) = 0.50
```

With `1`: the very first term is `0`, so `0 > epsilon` is false. The loop **never runs** and the program reports `sum = 0`.

`1.0` is a `double`, so the `int` denominator is converted and the division keeps the fraction.

Q25 · (c) Why `1.0` and not `1`?

`n * (n + 1)` is an `int`. With `1` on top, it's **integer division**: the fraction is thrown away.

`1 / (1 * 2)` $= 1/2 =$ **0**
`1.0 / (1 * 2)` $= 1.0/2 =$ **0.5**

```
1 / (n*(n+1)) with ints = 0,  1.0 / (n*(n+1)) = 0.50
```

With `1`: the very first term is `0`, so `0 > epsilon` is false. The loop **never runs** and the program reports `sum = 0`.

`1.0` is a `double`, so the `int` denominator is converted and the division keeps the fraction. Bonus: for very large n ($n > 46340$), `n * (n + 1)` itself overflows a 32-bit `int`. Writing `1.0 / ((double)n * (n + 1))` avoids that too.

Q25 · (d) How many iterations?

This series: the loop continues while

$$\frac{1}{n(n+1)} > \varepsilon \iff n(n+1) < \frac{1}{\varepsilon} \iff n \lesssim \sqrt{1/\varepsilon}$$

Q25 · (d) How many iterations?

This series: the loop continues while

$$\frac{1}{n(n+1)} > \varepsilon \iff n(n+1) < \frac{1}{\varepsilon} \iff n \lesssim \sqrt{1/\varepsilon}$$

so the iterations grow like $(1/\varepsilon)^{1/2}$: making ε 100× smaller costs **10**× more iterations.

Q25 · (d) How many iterations?

This series: the loop continues while

$$\frac{1}{n(n+1)} > \varepsilon \iff n(n+1) < \frac{1}{\varepsilon} \iff n \lesssim \sqrt{1/\varepsilon}$$

so the iterations grow like $(1/\varepsilon)^{1/2}$: making ε 100× smaller costs **10**× more iterations.

Terms $1/2^n$**:**
$1/2^n > \varepsilon \iff 2^n < 1/\varepsilon \iff n < \log_2(1/\varepsilon)$,
so only **logarithmic** growth: 100× smaller ε costs just ≈ 7 more iterations.

Measured (`q25_series.c`):

ε	$1/[n(n+1)]$	$1/2^n$
10^{-2}	9	6
10^{-4}	99	13
10^{-6}	999	19

Q25 · (d) How many iterations?

This series: the loop continues while

$$\frac{1}{n(n+1)} > \varepsilon \iff n(n+1) < \frac{1}{\varepsilon} \iff n \lesssim \sqrt{1/\varepsilon}$$

so the iterations grow like $(1/\varepsilon)^{1/2}$: making ε 100× smaller costs **10**× more iterations.

Terms $1/2^n$**:**
$1/2^n > \varepsilon \iff 2^n < 1/\varepsilon \iff n < \log_2(1/\varepsilon)$, so only **logarithmic** growth: 100× smaller ε costs just ≈ 7 more iterations.

Measured (`q25_series.c`):

ε	$1/[n(n+1)]$	$1/2^n$
10^{-2}	9	6
10^{-4}	99	13
10^{-6}	999	19

Matches the formulas: $\sqrt{10^6} = 1000$, $\log_2 10^6 \approx 19.9$.

Notice also: stopping at term $\leq \varepsilon$ leaves an error of $\frac{1}{N+1} \approx \sqrt{\varepsilon}$, *not* ε (0.999 for $\varepsilon = 10^{-6}$). Slowly converging series are both slow *and* less accurate than they look.

That's all 25

Top mistakes seen in this mock

- integer division (`1/2`, `total/n`)
- `=` vs `==`; `x^2` instead of `x*x`
- forgetting to initialize `sum`
- `<` vs `<=` at boundaries (Q7, Q13, Q14)
- dangling `else`, missing `break`
- array index 6 in an array of 6

Before the real exam

- Trace loops with a **table**, row by row.
- Test boundary inputs by hand.
- Write `%` specifiers carefully: `%d`, `%f`, `%lf`.
- Re-run every program in `examples/` and change the inputs.

TIP Good luck on the exam!